

ComplexGitSync 0000.04– User Guide

Nicolas Flipo

August 19, 2026

Contents

1	User Guide	2
2	CLI Overview	2
3	Lifecycle Contract	2
4	Commands	2
4.1	initialise	2
4.2	create-cgs	3
4.3	clean-init	3
4.4	purge	3
4.5	pull	3
4.6	pull-force	4
4.7	clone	4
4.8	checkout	4
4.9	add	4
4.10	commit	4
4.11	push	5
4.12	freeze-release	5
4.13	freeze	5
4.14	launch-release	5
4.15	view-tree	6
4.16	status	6
5	Document Formats	6
5.1	Local Authoring Spec (.cgs)	6
5.2	Repository Placement and Nested Discovery	8
5.3	Git Tree State Snapshot (.gts)	8
5.4	Planning / GOC Documents (.goc)	9
5.5	Local Git Register and Sync Ledger (.lgr)	9
6	Configuration and Environment	9
7	Error Reference	10
8	Troubleshooting	10

1 User Guide

This chapter is the authoritative reference for every user-facing command, configuration option, document format, and error condition exposed by `ComplexGitSync`.

2 CLI Overview

The single entry-point is the `cgitsync` command.

```
$ cgitsync <command> [options]
```

Running `cgitsync` without a command prints a help summary and exits with code 0.

3 Lifecycle Contract

This guide follows the simplified canonical lifecycle:

1. `initialise(.cgs)` → clone all repos → **READY** (*new project*)
`initialise(.gts)` → restore from snapshot → **READY** (*existing project*)
2. `pull(.cgs|.gts)` → resync an existing tree
3. `checkout(.gts)` / `add` / `commit` / `push` / `tag` → git operations on the tree
4. `freeze` → emit the next `.gts` id and update `<Project_name>.lgr`

The project-local `.lgr` register and sync ledger are both active (T24, T32). Every synchronisation operation is automatically recorded as an immutable DAG event in the `[[ledger]]` section of the `.lgr` file.

4 Commands

The CLI exposes two levels once a `GitTree` is **READY**. The minimalist level is `initialise`, `clean-init`, `freeze-release`, `freeze-release-force`, `status`, `view-tree`, and `launch-release`. The expert level exposes the individual operations: `branch`, `checkout`, `purge`, `validate`, `clone`, `pull`, `pull-force`, `add`, `commit`, `push`, `tag`, and `freeze`. Redundant inspection aliases such as `print`, `view-operation`, and `validate-topology` are not part of the public CLI surface.

4.1 initialise

```
$ cgitsync initialise <spec.cgs> [--output-path DIR]
$ cgitsync initialise <snapshot.gts>
$ cgitsync initialise --project NAME --repo PROVIDER:OWNER/REPO [--repo ...] [--output-path DIR]
```

Unified initialisation entry point. Dispatches on file extension:

- `.cgs` source (clone mode): clones the full repository tree. The optional `-output-path` controls `CGSPATH`; `CGSHOME` is derived as `CGSPATH/<project-name>`. When omitted, `CGSPATH` defaults to `../...` On success, ends in **READY**. If clone setup fails because stale generated state is still present, the CLI prints `Try clean-init method`.
- `.gts` source (restore mode): loads a saved snapshot directly. On success, ends in the lifecycle state recorded in the snapshot.

- Direct CLI authoring: requires `-project` and at least one repeatable `-repo`. It is mutually exclusive with a source file. The CLI collects values only and calls `ComplexGitSyncClient.configure()`; that reusable Python API delegates parsing, normalization, and validation to `cgs_format.py` and produces the same canonical `CgsDocument` as an equivalent `.cgs` file.

All initialization paths end in a `READY` tree or fail explicitly.

4.2 create-cgs

```
$ cgitSync create-cgs --project CGSil1 \
  --repo github:flipoyo/ComplexGitSync \
  --repo codeberg:GX4G/GX4G \
  --output CGSil1.cgs
```

Builds the canonical `CgsDocument` through the offline format pipeline and serializes concise `.cgs` TOML. It performs no Git or network work.

4.3 clean-init

```
$ cgitSync clean-init <spec.cgs> [--output-path DIR]
```

Recovery-oriented initialisation for a `.cgs` source. It follows the same high-level pipeline as `initialise(.cgs)`, but inserts a cleanup phase before cloning:

load->expand->validate->purge->clone

Use it when an earlier initialisation attempt left partial child checkouts, stale submodule metadata, or an obsolete root `.gitmodules` file.

4.4 purge

```
$ cgitSync purge <spec.cgs> [--output-path DIR]
```

Removes generated clone state from the resolved `CGSHOME` without cloning. It deletes child repository directories declared directly under the project root, project-local `*.lgr` files, and the root `.gitmodules` file. Nested directories that are not immediate root children are left untouched by this command.

4.5 pull

```
$ cgitSync pull [spec.cgs|snapshot.gts]
```

Resynchronises the project tree. `.cgs` sources reload the source, discover nested configs, then resynchronise the loaded tree. `.gts` sources load the saved registry as the starting point and then run the same pull workflow. In both cases the operation starts at the project root: `root -> parent -> leaf`. The root repository receives `git pull -ff-only`; each child repository is then updated through its parent submodule link. Use `launch-release` when you want to check out a frozen recorded release state instead of pulling current remotes. If local changes block this safe pull, the CLI prints `You can try cgitSync pull-force command`.

4.6 pull-force

```
$ cgitsync pull-force [spec.cgs|snapshot.gts]
```

Destructively resynchronises the same tree. The root repository is forced to the selected remote branch with `git fetch`, `git checkout -B <branch> FETCH_HEAD`, and `git clean -fd`; child submodules are then force-updated parent-first. This command can discard local uncommitted changes and untracked files.

4.7 clone

```
$ cgitsync clone <spec.cgs> [--target-dir DIR] [--output-path DIR]
```

Clones the full repository tree declared by the `.cgs` file. This CLI command delegates to `ComplexGitSyncClient.clone`. When `-target-dir` is omitted, the project root is derived from the specification. `-output-path` provides the parent directory used to derive the project root.

For each repository, `cgitsync` tries the repo target branch first and falls back to the repo fallback branch when the preferred branch does not exist on the remote. Nested `.cgs` files are discovered after parent repositories are cloned, so explicit and automatic nested trees are materialised during the same command.

4.8 checkout

```
$ cgitsync checkout <branch> [--gts <snapshot.gts>] [--ref-kind branch|tag]
```

Checks out a branch or tag across the whole tree, parent-first. Loads the READY registry from `-gts`, then propagates the ref, creates missing local branches, and runs `git checkout` in every repo. On success the tree remains READY and a new `.gts` snapshot is written.

4.9 add

```
$ cgitsync add [--gts <snapshot.gts>] [--dry-run]
```

Stages all changes across the tree (`git add -all`), leaf-first. Requires READY; tree stays READY. With `-dry-run`, `cgitsync` prints the leaf-first execution plan and does not mutate repositories.

4.10 commit

```
$ cgitsync commit <message> [--gts <snapshot.gts>] [--no-stage] [--dry-run]
```

Commits changes across the tree, leaf-first. Loads the READY registry from `-gts`. Repositories with no staged changes after the optional `git add -all` step (suppressed by `-no-stage`) are silently skipped. Requires a READY tree; the tree remains READY on success. Commit preflight now emits warnings for actionable workspace issues such as dirty worktrees, ahead branches, stale recorded `commit_sha` values, or child repositories that are not linked as submodules. With `-dry-run`, `cgitsync` prints the planned stage/commit sequence without performing git writes.

The commit message conventionally ends with `CGS#VERSION`.

4.11 push

```
$ cgitssync push [--gts <snapshot.gts>] [--dry-run]
```

Pushes all repositories to their configured remotes, leaf-first. Loads the READY registry from `-gts`. Each repo is pushed to `entry.remote_name` (defaulting to "origin") on its currently resolved branch. Refreshes the stored hash in `GitTree`. Requires a READY tree; the tree remains READY on success. Push preflight blocks detached HEADs, missing remotes, unresolved merges, behind/diverged branches, and missing submodule links; it warns when the local branch is ahead or when the loaded snapshot records an older `commit_sha`. With `-dry-run`, `cgitssync` prints the push plan without mutating any repository.

4.12 freeze-release

```
$ cgitssync freeze-release <name> <message> [--gts <snapshot.gts>] [--dry-run]
$ cgitssync freeze-release-force <name> <message> [--gts <snapshot.gts>] [--dry-run]
```

Minimalist release workflow from a READY tree. It chains public operations in order: `add -> commit -> pull -> push -> freeze`. The force variant uses `pull-force` instead of `pull`. The release name becomes the shared tag and freeze snapshot label; the message is used for the release commit.

4.13 freeze

```
$ cgitssync freeze <name> [--gts <snapshot.gts>] [--dry-run]
```

Performs a release freeze across all repos, leaf-first: stage/commit, create+push the shared release tag, refresh SHA/state, and write a `.gts` snapshot. Loads the READY registry from `-gts`. Requires READY and leaves the tree READY. Freeze preflight checks remote availability, branch alignment, detached HEADs, unresolved merges, tag absence, and submodule links for all parent/child repository boundaries. Dirty worktrees and stale recorded `commit_sha` values are reported as warnings because freeze will stage, commit, and snapshot them. Freeze snapshots also include a deterministic `freeze_manifest` section that records freeze invariants (immutable snapshot, validated workspace, synchronized tag, `release-name`, ledger checkpoint) and declares `launch_state` as the canonical restore operation. Their immutable snapshot file-name includes the release, for example `gts-000001-release-2026.05.gts`, while the ledger id remains `gts-000001`. With `-dry-run`, `cgitssync` prints the planned add/commit/tag/push graph for the loaded workspace without mutating it.

4.14 launch-release

```
$ cgitssync launch-release <name> [--gts <snapshot.gts>]
```

Checks out the frozen release tag `<name>` across the loaded READY tree, parent-first, and writes a fresh `.gts` snapshot. This is the release oriented CLI path for returning a workspace to a frozen version; direct tag creation remains available from the Python client API as `tag()`.

The integration suite validates local no-network lifecycle coverage through the CLI path, including initialisation and the READY git command cycle `add -> commit -> push -> freeze -> launch-release`.

4.15 view-tree

```
$ cgitasync view-tree [spec.cgs|snapshot.gts] [--depth N] [--collapse  
  REPO]
```

Renders a deterministic terminal tree that focuses on repository topology and operational state: `name [branch] local_state sync_state`. Use `-depth` to limit visible levels and repeat `-collapse` to hide selected subtrees.

Branch Topology Propagation Rules (T35):

1. **Reference branch:** The root repository's current branch is the canonical reference for all repos.
2. **Leaf-to-root inheritance:** Branch targeting flows root-first via `propagate_global_branch`. `validate-topology` verifies the on-disk state without issuing any git writes.
3. **Allowed divergence:** Repositories in a frozen (TAG) state produce a `tag_divergence` diagnostic that does *not* make the topology incoherent.
4. **Incoherent states (blocking):** `misaligned_branch` (different branch from root) and `detached_head` (detached HEAD without a tag reference).

4.16 status

```
$ cgitasync status [--gts FILE]
```

Summarises tree readiness and per-repository sync state. The table reports the local branch, upstream branch, local worktree state, ahead/behind tracking counts, current HEAD, and the commit recorded in the loaded `.gts`.

5 Document Formats

`.cgs` means **ComplexGitSync** specification, `.gts` means **GitTreeState** snapshot, `.goc` means **GitOrchestrationCommand** plan, and `.lgr` means **Local Git Register**.

5.1 Local Authoring Spec (.cgs)

A TOML file that describes the repository tree to manage. It is the only hand-authored input; it is never modified at runtime. The normal form has two top-level keys:

```
project = "CGSi11"  
repos = [  
  "gitlab:CGS_test/CGSi11",  
  "gitlab:CGS_test/CGSi12",  
  "github:flipoyo/CGSi11",  
]
```

Repository identifiers have the form `provider:owner/repository`. Nested GitLab namespaces are supported; the final path segment is the repository name. The document pipeline is:

PARSE -> NORMALIZE -> VALIDATE -> canonical CgsDocument

The canonical providers and hosts are:

Provider	Host	Remote generation
github	github.com	SSH and HTTPS
gitlab	gitlab.com	SSH and HTTPS
codeberg	codeberg.org	SSH and HTTPS
custom	explicit URL required	SSH and HTTPS

Repository-ID parsing is provider-independent and deterministic. For example:

```
codeberg:GX4G/GX4G
```

normalizes to `gitprovider = codeberg`, `project_owner_name = GX4G`, and `repo_name = GX4G`. A custom identifier must use an inline table so the host is explicit:

```
{ repository = "custom:team/tool", gitprovider_url = "https://git.
  example.com" }
```

No host is inferred for `custom`. The configured access protocol then selects `git@host:owner/repository.git` or `https://host/owner/repository.git`.

The textual grammar and its only parser, `parse_repo_id()`, live in `cgs_format.py`. The provider model in `git_repo.py` receives the already-separated provider, owner, and repository fields and composes remotes; it does not parse authoring shorthand.

The `.cgs` format pipeline is deterministic and offline. A syntactically valid repository may therefore reference a host, owner, branch, or tag that is not currently reachable. Remote existence and reference availability are checked only by explicit Git operations during runtime resolution and execution.

Normalization supplies `default_branch = main`, makes `fallback_branch` equal to that default, selects `access_protocol = ssh`, enables `nested_config = auto`, and uses the repository name as `relative_path`. If exactly one repository name matches `project`, its path is inferred as `..`.

Advanced inline tables are used only where a value differs from those defaults:

```
project = "demo"
repos = [
  "github:owner/demo",
  { repository = "github:owner/docs", relative_path = "documentation",
    nested_config = "disabled" },
]
```

The legacy advanced `[project]` and `[[repos]]` tables remain accepted for project-wide branch settings, custom provider URLs, tags, and other explicit overrides. Unknown providers, malformed identifiers, duplicates, and missing `project` or `repos` are rejected before a `GitTree` is built.

Two checked-in files document this boundary. Copy `examples/template.cgs` for normal authoring. Its equivalent `examples/normalized_template.cgs` expands every inferred field and is provided as a developer reference for the canonical `CgsDocument` shape. The two files are regression-tested for semantic equivalence.

Generated specifications use the same concise representation. The `GitTree.to_cgs()` convenience method delegates model projection to `cgs_format.py`; neither `GitTree` nor `GitRepo` formats TOML or implements the authoring grammar. Serialization omits every default that normalization can deterministically reconstruct and retains inline tables only for exceptional settings such as a non-default branch, path, protocol, tag, custom host, or discovery rule.

The supported round trip is semantic:

```
.cgs -> CgsDocument -> GitTree -> CgsDocument -> .cgs
```

Parsing and normalizing the generated file must reproduce the initial canonical document. Whitespace, comments, and TOML table layout need not be byte-for-byte identical.

5.2 Repository Placement and Nested Discovery

For every non-root entry, `relative_path` is relative to the parent repository represented by the current `.cgs`; it is not relative to the process working directory. At the top level that parent is the project root. While expanding a nested config, it is the repository containing that config. The default is the repository name. A unique repository matching the project name uses `.` because it represents the current root.

The `nested_config` option controls whether traversal continues after the repository is available:

- `auto` (default) loads the sole root-level `*.cgs` file, if one exists; multiple matches are rejected as ambiguous.
- `disabled` does not inspect the repository for nested config.
- A relative path ending in `.cgs` loads that exact file inside the repository. It may not escape the repository root.

For the integration topology, `CGSil2.cgs` contains:

```
{ repository = "github:flipoyo/CGSih1", relative_path = "../CGSih1",  
  nested_config = "disabled" }
```

Because the config describes the tree below `CGSil2`, the path resolves from that directory:

```
<CGSHOME>/CGSil2/../CGSih1 -> <CGSHOME>/CGSih1
```

It therefore references the existing sibling declared by `CGSil1.cgs`, instead of cloning another `CGSih1` under `CGSil2`. Discovery recognizes the already-registered absolute path and keeps the canonical entry. The cross-reference is `disabled` because the canonical root-level `CGSih1` entry is responsible for loading `CGSih1.cgs`.

5.3 Git Tree State Snapshot (`.gts`)

A TOML file generated automatically by bootstrapping and release operations. It must never be hand-edited. It captures the exact commit SHAs and lifecycle state of the tree at a point in time.

Top-level sections:

- `[document]` — `format_version`, `schema_version`, `generated_at`, `command_origin`, `hash_algorithm`, `snapshot_hash`.
- `[project]` — project name and `root_absolute_path`.
- `[tree_state]` — `lifecycle_state`, `is_ready`, `registry_complete`.
- `[[repo_state]]` — one entry per repo; includes `commit_sha`, `sync_state`, and ref information.

Why both version fields and hash metadata exist:

- `format_version` identifies the broad document family and keeps long-range compatibility intent explicit.
- `schema_version` identifies the exact field-level contract used by validation and canonical snapshot serialization.

- `snapshot_hash` (with `hash_algorithm = "sha256"`) provides deterministic workspace identity: identical tree state produces identical digest even if volatile metadata (`generated_at`, `command_origin`) differs.

How this is used at runtime:

- During snapshot generation, `ComplexGitSync` writes `schema_version`, `hash_algorithm`, and canonical `snapshot_hash`.
- During load/validation, `snapshot_hash` is recomputed from the canonical payload and must match.
- The project-local `.lgr` register uses this canonical hash to deduplicate equivalent snapshots.

5.4 Planning / GOC Documents (`.goc`)

Used for project identity and planning metadata. Supported in TOML, YAML, or JSON. YAML support requires the optional PyYAML dependency.

5.5 Local Git Register and Sync Ledger (`.lgr`)

A TOML file maintained per project at `<project-root>/<Project_name>.lgr`. It is updated automatically whenever a `.gts` snapshot is written.

Sections:

- `[register]` — current snapshot pointer (`current_snapshot_id`, `current_snapshot_hash`, `current_snapshot_path`).
- `[[snapshots]]` — list of stable `gts-XXXXXX` identifiers, one per unique workspace state, deduplicated by SHA-256 hash.
- `[[ledger]]` — append-only list of synchronisation events forming a DAG via `parent_sync_ids`.

Ledger event schema:

```
[[ledger]]
sync_id      = "lgr-000001"
parent_sync_ids = []
operation    = "clone"
timestamp    = "2026-05-20T19:48:50.159Z"
actor       = "username"
workspace_hash = "5f7c8a2d..." % 64-character SHA-256 hex digest
gts_snapshot_id = "gts-000001"
affected_repos = ["demo", "dep-a"]
```

The `workspace_hash` field equals the canonical SHA-256 digest (`document.snapshot_hash`) of the associated `.gts` file, creating a direct link between each ledger event and the immutable workspace state it records. Events are parents-before-children in DAG order.

6 Configuration and Environment

`ComplexGitSync` uses project files for topology and does not require global topology configuration. Snapshot discovery can use `CGSHOME`; when it is not set, the `initialise(.cgs)` output-path default is `CGSPATH=../..`; later commands can also walk upward from the current directory to find `.cgitsync/state/`. The logging and runtime-state subsystems use the user state directory by default:

- logs: `/.local/state/ComplexGitSync/logs/`
- latest runtime-state mapping: `/.local/state/ComplexGitSync/runtime/`

If `XDG_STATE_HOME` is set, these directories are rooted there instead.

7 Error Reference

ConfigValidationError Raised when `project` or `repos` is missing, a repository identifier is malformed, or a canonical field is invalid (for example an unknown `gitprovider`).

GitSyncError Raised when clone-time Git operations fail, for example because the remote branch cannot be resolved or the target directory is already occupied.

Exit code 1 General CLI failure (parse error, validation error).

Exit code 2 Command exists but is not yet implemented.

8 Troubleshooting

“gitprovider_url is required for custom provider” Add a `gitprovider_url` field to the offending `[[repos]]` entry in your `.cgs` file.

validate exits non-zero despite a correct-looking file Check that `project` is present and every `repos` entry uses `provider:owner/repository`. For advanced `[[repos]]` blocks without a `repository` shorthand, include `project_owner_name` and `project_name`.

Clone destination already exists and is not empty Re-run `clone` with `-target-dir` to force a clean location, or let `cgitsync` choose a suffixed default directory.

No cloneable branch found for <repo> Confirm that the remote exposes either the repo’s `default_branch` or its `fallback_branch`.

PyYAML import error when using .goc files Add PyYAML in the Pixi environment: `pixi add pyyaml`.